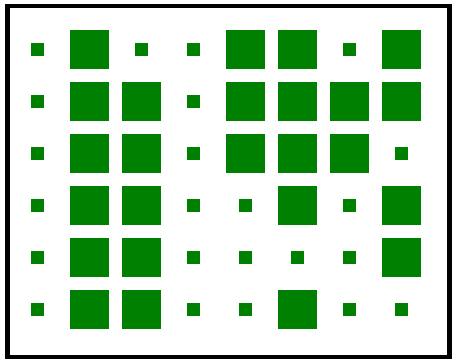
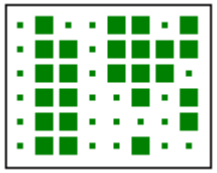




Lab 3
B-Minor Lexical Analyzer

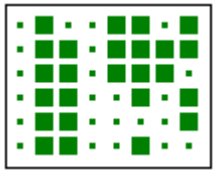
CS 330:
Programming Languages
And Compilers

Process to Explore B-Minor



1) Review B-Minor at a high level – accounting for common imperative language features (variables, types, decisions, loops, functions…)
2) Write a small B-Minor program that uses most language features
3) Brainstorm tokens
4) Begin compiling a list of questions about the language's syntax – aspects that are unclear or seem undefined

Obtain the Lab 3 Starting Code



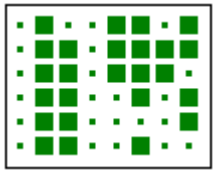
- Open a terminal and change directory into your **`cs330-IndividualLabWork`** directory
- Use git to obtain the lab 3 code:

```
git pull
```

- The directory **`lab3-B-Minor-Scanner`** should be added to your project
- Change directory into the lab3-B-Minor-Scanner directory

```
cd lab3-B-Minor-Scanner
```

lab3-B-Minor-Scanner Files Provided



`Makefile` → script to automate the code generation and compilation process

`B-Minor-Tokens.csv` → example lexeme and token definitions for B-Minor

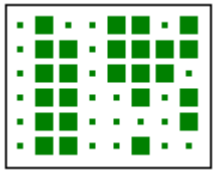
Special situations:

- Token with no regex (no rule)
- Lexeme with no token: no return value

`test-program.b` → b-minor program to scan

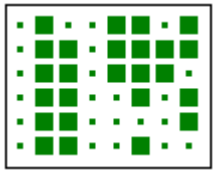
`lexer-result.txt` → output test scanner should produce when using `test-program.b` as input

Lab Assignment (1 of 7)



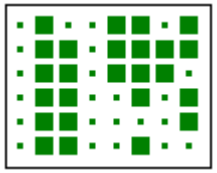
- **Goal: Students create a B-Minor scanner**
  - Must build upon the regular expressions, token names, and token values as supplied in the **`B-Minor-Tokens.csv`** file
  - You will create additional tokens and regular expressions
    - The `[ \t\n]` regex is used to skip spaces, tabs, and new lines. Action contains ***no return statement***, causes the scanner to keep going
    - `TOKEN_NO_ELSE` **is not used** in any action (it has no regular expression). We need it later for the grammar – should be in the enum
    - We'll deal with comments later

Lab Assignment (2 of 7) (Exercise 1)



- Add your files for this lab to the `lab3-B-Minor-Scanner` directory
- The analyzer must be defined using Flex
  - The Flex configuration file must be named **`b-minor-scanner.l`**
  - It may <u>not</u> contain a `main()` function
  - It may <u>not</u> contain the `enum` of token constants

Lab Assignment (3 of 7) (Exercise 2)

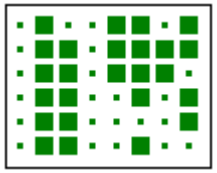


- Create a separate file containing the enum of token constants
  - Name the file **`tokens.h`**
  - The flex configuration, **`b-minor-scanner.l`**, will include the **`tokens.h`** file in the C code block at the top of the file

```
%{
    #include "tokens.h"
%}
```

    - Don't forget the `noyywrap` option

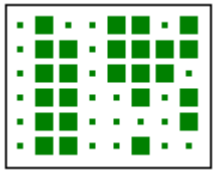
Lab Assignment (4 of 7) (Exercise 3)



- Write a separate function to display token names
  - Place the function in the file **`utils.c`**
  - Name the function **`get_token_name`**
    - Takes the token constant as a parameter
    - Returns matching token name (string)

      ```
      char *get_token_name(int token_value)
      ```
  - Include **`tokens.h`**
  - Create a **`utils.h`**

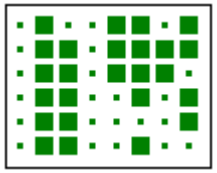
Lab Assignment (5 of 7) (Exercise 4)



How to test?

- Write a **`main`** function to run the scanning process
  - Place main() in the file **`test-scanner.c`**
  - Include **`utils.h`**
  - When run, **`main()`** repeatedly calls **`yylex()`**, printing the token name on the screen
    - What is the **`yylex()`** return value for EOF?
    - Use **`get_token_name()`** to get the name
    - printf: `%s` is a string
  - Example output is shown on the next slide

Example Output from the Test Scanner

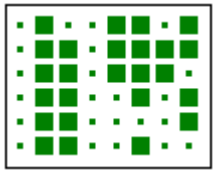


```
TOKEN_IDENT
TOKEN_COLON
TOKEN_BOOLEAN
TOKEN_SEMI
TOKEN_IDENT
TOKEN_COLON
TOKEN_STRING
TOKEN_ASSIGN
TOKEN_LITERAL_STRING
TOKEN_SEMI
```

(all of the expected output when scanning the test B-Minor program is found in **`lexer-result.txt`** file)

If your output does not match, your scanner is not working correctly, and you have not completed the lab

Lab Assignment (6 of 7)



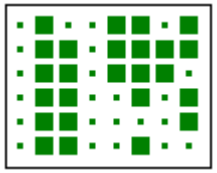
- To build your scanner use the `make` utility
  - In the lab3-B-Minor-Scanner directory, type:

    `make`

    - It will run Flex and compile the sources
- To test, use the **`test-program.b`** file

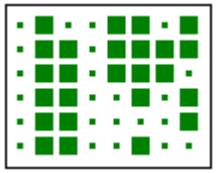
  `./a.out < test-program.b`
- To clean up generated files: **`make clean`**
- **Congratulations**, your compiler journey has begun in earnest! You have a lexical analyzer!

Lab Assignment (7 of 7)



- **Reflection: Review the files you created**
- Update the **`README.md`** file in the lab 3 directory and respond to the following questions:
  1) Explain the logic of the regular expressions for a character and string
  2) List and explain the purpose of each file you created or edited
  3) Explain the dependencies between the files you created or edited
  4) Explain the required changes to all these files if a new keyword or operator is added to the B-Minor language

Beginning and End of Work Session



- Using the terminal, in the cs330-IndividualLabWork directory, use the following commands:

  `git add .`

  - Note the trailing dot (preceded by a space) is required

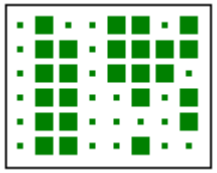
  `git commit -m "`*`A MEANINGFUL COMMENT`*`"`

  `git push`

  - The first time you push you may be asked to set the default branch. Enter the command as given in the message.

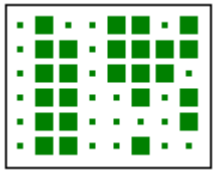
Submitting Your Lab Work

Submitting the Lab (1 of 2)



- Make sure to follow the description of how to update your git repository each time you update your files (page 13)
- The following page describes the submission process

Submitting the Lab (2 of 2)

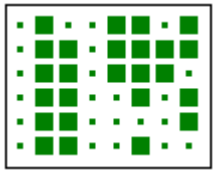


- When you have completed the lab (including following all steps on page 13 after your last changes are made), create and push an annotated tag to indicate the assignment is completed (the assignment isn't submitted until this is done):

```
git tag -a Lab3Submission1 -m "Lab completed"
git push --follow-tags
```

- You should receive an email within an hour verifying the lab was received for grading.

Errors When Using Git



- If an error occurs please capture a screenshot of the error and send that to the instructor along with a overview of what you were trying to do
  - Generally the errors are related to not being in the correct directory when using the Git commands